ACM PODC 2019 Tutorial

# From Classical to Blockchain Consensus: What Are the Exact Algorithms?

Y. Annie Liu

Scott Stoller

Computer Science Department
Stony Brook University, State University of New York

# Where is Stony Brook University?



# Many thanks!

great works by fabulous PhD students

Bo Lin: DistAlgo compiler, runtime system
Saksham Chand: Formal verification of Paxos
Jon Brandvein: Automatic incrementalization
Michael Gorbovitski: Automatic incrementalization
Tom Rothamel: Automatic incrementalization








insightful discussions with wonderful colleagues

on many intriguing algorithms in distributed systems and blockchain worlds

NSF and ONR grants

# Outline

Distributed consensus problems: classical, Byzantine, blockchain

Overview of well-known algorithms: ... Paxos ... Bitcoin ...

Method and language: DistAlgo, high-level, precise, executable

Exact algorithms: Paxos and Bitcoin

Demo of direct execution: only need Python installed

# Distributed systems

a set of processes, or agents, parties, players, entities, ...

- each has private memory, no shared memory
- they communicate by message passing, with delays

your experience with distributed systems

# Kinds of applications

- real-world distribution
  - — separate data storage, remote information source
  - — separate computing units, processing entities

  examples: corporate management, air traffic control, mobile apps, games, ..., general collaborative or competitive activities: auction, election, ...
- better quality of life
  - — enhancing reliability: replication, backup
  - — increasing performance, scalability: parallelization

  examples: cloud services, HPC, ..., replicated state machines, ...
- ease of modeling
  - — conceptual separation of data
  - — conceptual separation of processing

  examples: modular problem solving, ..., OO programming, ...

# Age of distributed computing

telecomm industry

information on Web

search engines

social networks

cloud computing

mobile computing

⋮

distributed systems are increasingly important and challenging

at the core are distributed algorithms

# Example problems needing algorithms

distributed mutual exclusion
leader election
distributed consensus
atomic commitment
replication
clock synchronization
distributed file system
distributed database
snapshot
termination detection
deadlock detection
failure detection
distributed garbage collection
routing
broadcast
⋮
almost all having to do with "consensus"

# Example: Distributed mutual exclusion

problem:

multiple processes access a shared resource,
and need to access it mutually exclusively,
in what is called a critical section (CS),
i.e.,
no two processes are in CS at the same time.

- how do processes communicate?
- can processes or communications fail?
- correctness properties?
- efficiency measures?

# Example: Distributed consensus

problem:

multiple processes agree on a value,
or a sequence of values,
essential for replicated services.

an easier problem than distributed mutual exclusion

- same basic questions, and more can be asked
- how can processes or communications fail?
- could it be impossible to solve at all?

# Distributed consensus

what's to be achieved:

multiple processes agree on a value,

or a sequence of values,

or: partial order, k-set consensus, epsilon agreement, ...

why it's essential for distributed services: at Amazon, Google, ...

- a service must use replication to tolerate failures:

  e.g., machine crashes, network outages, ...

- replicated processes must agree on the state of the service or the sequence of operations that have been performed:

  e.g., customer order has been placed, paid, but not yet shipped, ...

- so when some processes fail,
  the remaining processes can continue to function correctly

# Failures in distributed systems

process failure: asynchrony

processes can crash, later recover, or be too slow ↕

link failure: asynchrony

links can drop, delay, reorder, or duplicate messages

Byzantine failure:

processes/links can behave arbitrarily: misbehave or attack

Sybil attack:

process identities can be pseudonyms, i.e., false names

# Distributed consensus problems

multiple processes agree on a value, or a sequence of values,

under different failure models:

- classical consensus: crash failures
- Byzantine consensus: Byzantine failures
- blockchain consensus: Sybil attack

# Correctness and efficiency

correctness criteria:

- agreement: safety
  all non-faulty processes must decide on the same value or sequence of values
- validity: safety
  the value or values must be proposed by some process
- termination: liveness
  all non-faulty processes eventually decide

complexity measures:

- message complexity
- round complexity how live

# Impossibility result

In asynchronous distributed systems, consensus is impossible, even if only one process can have a crash failure.

Fischer, Lynch, Patterson, JCAM 1985

Impossibility of distributed consensus with one faulty process

However, in reality, this worst case is highly unlikely.

There is a degree of randomness.

Do you want randomness?

How much?

# More results

| failure | synchronous system | asynchronous system |
|---|---|---|
| no | agreement attainable<br><br>common knowledge too | agreement attainable<br><br>concurrent common knowledge too |
| process | agreement attainable<br>$\geq f + 1$ rounds | agreement not attainable<br>(FLP 1985) |
| crash | $f < n$ processes | |
| Byzantine | $3f < n$ processes | |
| Sybil | ? | |

# Kinds of algorithms

many variants at high-level:

- synchronous vs. asynchronous
- crash failures vs. Byzantine failures vs. Sybil attacks
- deterministic vs. randomized vs. probabilistic

even more variants at lower-level:

- primary-backup, master-slave, vs. peer-to-peer, gossip
- majority vs. quorum vs. committee
- permissioned vs. permissionless
- proof-of-work vs. proof-of-stake

even much more variants at even lower-level:

- failure detectors, stable storage, garbage collection, ...

# Outline

Distributed consensus problems: classical, Byzantine, blockchain

Overview of well-known algorithms: ... Paxos ... Bitcoin ...

Method and language: DistAlgo, high-level, precise, executable

Exact algorithms: Paxos and Bitcoin

Demo of direct execution: only need Python installed

# Distributed consensus algorithms

1. classical consensus algorithms:
   Virtual Synchrony (VS), Viewstamped Replication (VR),
   Paxos, ...

2. Byzantine consensus algorithms:
   Pratical Byzantine Fault Tolerance (PBFT), ...

3. blockchain consensus algorithms:
   Bitcoin, ...

# 1. Classical consensus algorithms

key ideas: coordinator/primary/leader, group/quorum/majority
practice: 3 processes/replicas, $\leq$1 faulty; or 5 total, $\leq$2 faulty

Virtual Synchrony:
maintain group membership for reliable communication

Viewstamped Replication:
primary-backup, with normal operation and recovery

Paxos, Paxos made simple, ...:
parliament for distributed DB; proposers-acceptors-learners

with Failure Detectors:
& stable storage; clean pseudocode, conditions, complexities

others: Chubby, Paxos made live, Paxos for system builders, ...,
Raft, Paxos made moderately complex

# 2. Byzantine consensus algorithms

better known as Byzantine Fault Tolerance (BFT)

key ideas:

- more non-faulty processes over faulty processes
  3f+1 vs. 2f+1 w/o Byzantine, e.g., 4 procs,1 f; or 7,2
  before: wait for f+1
  here: wait for 2f+1, take f+1 from 2f+1

- process authentication using signed messages
  use cryptographic functions for signing and verification
  – public key: easy but expensive
  – shared key: efficient but more complex

algorithms and variants:
  Practical BFT (PBFT)
  FaB, Zyzzyva, ... Abstract, ... Byzantizing Paxos, ...
  Prime: guarantees performance under attack

# 3. Blockchain consensus algorithms

key ideas:

- overcome pseudonyms with hard proof requirements
  Bitcoin proof of work (PoW): miners solve a hard puzzle
- no leader or centralized authority decides blocks for others
  solve first and become leader, add block and gossip about it
- but, a lot of compromises to address serious problems
  huge cost: proof of stake (PoS), trusted quorum slices, different cryptos, different puzzles, ...
  chance of reversing transactions: leader do small few commits, committee/council rotate and do BFT, ...

algorithms and variants:

Bitcoin PoW, PoS, Bitcoin-NG, Byzcoin, Stellar (SCP), Hashgraph, Algorand, Avalanche, ...
distributed DB advertised as blockchain: Hyperledger, ...

# Blockchain vs. classical consensus

defend against Sybil attack:

– huge cost

– large computing power needed

– low throughput

+ good scalability in number of processes

– chance of reversing the transaction

– change of creating a divergence, or fork

hot:

a lot of new ideas and use (or reinvention) of ideas for classical consensus

# Classical consensus algorithms/variants (I)

Group Membership

Primary-Backup

Paxos

Failure Detectors

| | Name | Description | Language used |
|---|---|---|---|
| 1 | VS-ISIS | Reliable group communication, Birman-Joseph 1987 | English (items) |
| 2 | VS-ISIS2 | Virtual synchrony, Birman-Joseph 1987 | English |
| 3 | EVS | Extended Virtual Synchrony for network partition, Amir et al 1995 | pseudocode |
| 4 | Paxos-VS | Virtually synchronous Paxos, Birman-Malkhi-van Reness 2012 | pseudocode |
| 5 | VR | Viewstamped replication, Oki-Liskov 1988 | pseudocode (coarse) |
| 6 | VR-Revisit | VR revisited, Liskov 2012 | English (items) |
| 7 | Paxos-Synod | Paxos in part-time parliament, Lamport 1998 | TLA (single-value) |
| 8 | Paxos-Basic | Single-value Paxos, Lamport 2001 | English (items) |
| 9 | Paxos-Fast | Single-value Paxos with replicas proposing, Lamport 2006 | English (items), TLA+ |
| 10 | Paxos-Vertical | Single-value Paxos with external starting of leader election, Lamport-Malkhi-Zhou 2009 | PlusCal |
| 11 | ACT | Single-value consenus with failure detection, Aguilera-Chen-Toueg 2000 | pseudocode |
| 12 | ACT-Store | ACT with stable storage, same reference as above | pseudocode |

thanks to Saksham Chand for beautiful colored frames!

# Classical consensus algorithms/variants (II)

Formally specified algos

More variants

Formally specified algos

| | | | |
|---|---|---|---|
| 13 | Paxos-Time | Paxos with time analysis, De Prisco-Lampson-Lynch 2001 | IOA (single-value) |
| 14 | Paxos-PVS | Single-value Paxos for proof, Kellomäki 2004 | PVS |
| 15 | Chubby | Paxos in Google's Chubby lock service, Burrows 2006 | English (partial items) |
| 16 | Chubby-Live | Chubby in Paxos made live, Chandra-Griesemer-Redstone 2007 | English |
| 17 | Paxos-SB | Paxos for System Builders, Kirsch-Amir 2008 | pseudocode |
| 18 | Mencius | Paxos with leaders proposing in turn, Mao et al 2008 | English (items) |
| 19 | Zab | Yahoo/Apache's Zookeeper atomic broadcast, Junqueira-Reed-Serafini 2011 | English (items) |
| 20 | Zab-FLE | Zab with Fast Leader Election (FLE) and pseudocode, Medeiros 2012 | pseudocode |
| 21 | EPaxos | Egalitarian Paxos, Moraru-Andersen-Kaminsky 2013 | pseudocode |
| 22 | Raft | Consensus in RAMCloud, Ongaro-Ousterhout 2014 | pseudocode |
| 23 | Paxos-Complex | Paxos made moderately complex, van Reness-Altinbuken 2015 | pseudocode, Python |
| 24 | IronRSL | Paxos in Microsoft's IronFleet for proof, Hawblitzel et al 2015 | Dafny |
| 25 | Paxos-TLA | Paxos for proof using TLAPS, Chand-Liu-Stoller 2016 | TLA+ |
| 26 | LastVoting-PSync | Single-value Paxos in Heard-Of model for proof using PSync, Drăgoi-Henzinger-Zufferey, 2016 | PSync |
| 27 | Raft-Verdi | Raft for proof using Coq, Wilcox-Sergey-Tatlock 2017 | Verdi |
| 28 | Paxos-EPR | Paxos in effectively propositional logic for proof, Padon et al 2017 | Ivy |
| 29 | Paxos-Decon | Paxos deconstructed, Garcia et al 2018 | Scala/Akka |
| 30 | Paxos-High | Paxos in high-level executable specification, Liu-Chand-Stoller 2019 | DistAlgo |

thanks to Saksham Chand for beautiful colored frames!

# Basic Paxos, Lamport's description

[Lam01-SIGACT News]

Putting the actions of the proposer and acceptor together, we see that the algorithm operates in the following two phases.

**Phase 1.** (a) A proposer selects a proposal number $n$ and sends a $prepare$ request with number $n$ to a majority of acceptors.

(b) If an acceptor receives a $prepare$ request with number $n$ greater than that of any $prepare$ request to which it has already responded, then it responds to the request with a promise not to accept any more proposals numbered less than $n$ and with the highest-numbered proposal (if any) that it has accepted.

**Phase 2.** (a) If the proposer receives a response to its $prepare$ requests (numbered $n$) from a majority of acceptors, then it sends an $accept$ request to each of those acceptors for a proposal numbered $n$ with a value $v$, where $v$ is the value of the highest-numbered proposal among the responses, or is any value if the responses reported no proposals.

(b) If an acceptor receives an $accept$ request for a proposal numbered $n$, it accepts the proposal unless it has already responded to a $prepare$ request having a number greater than $n$.

...

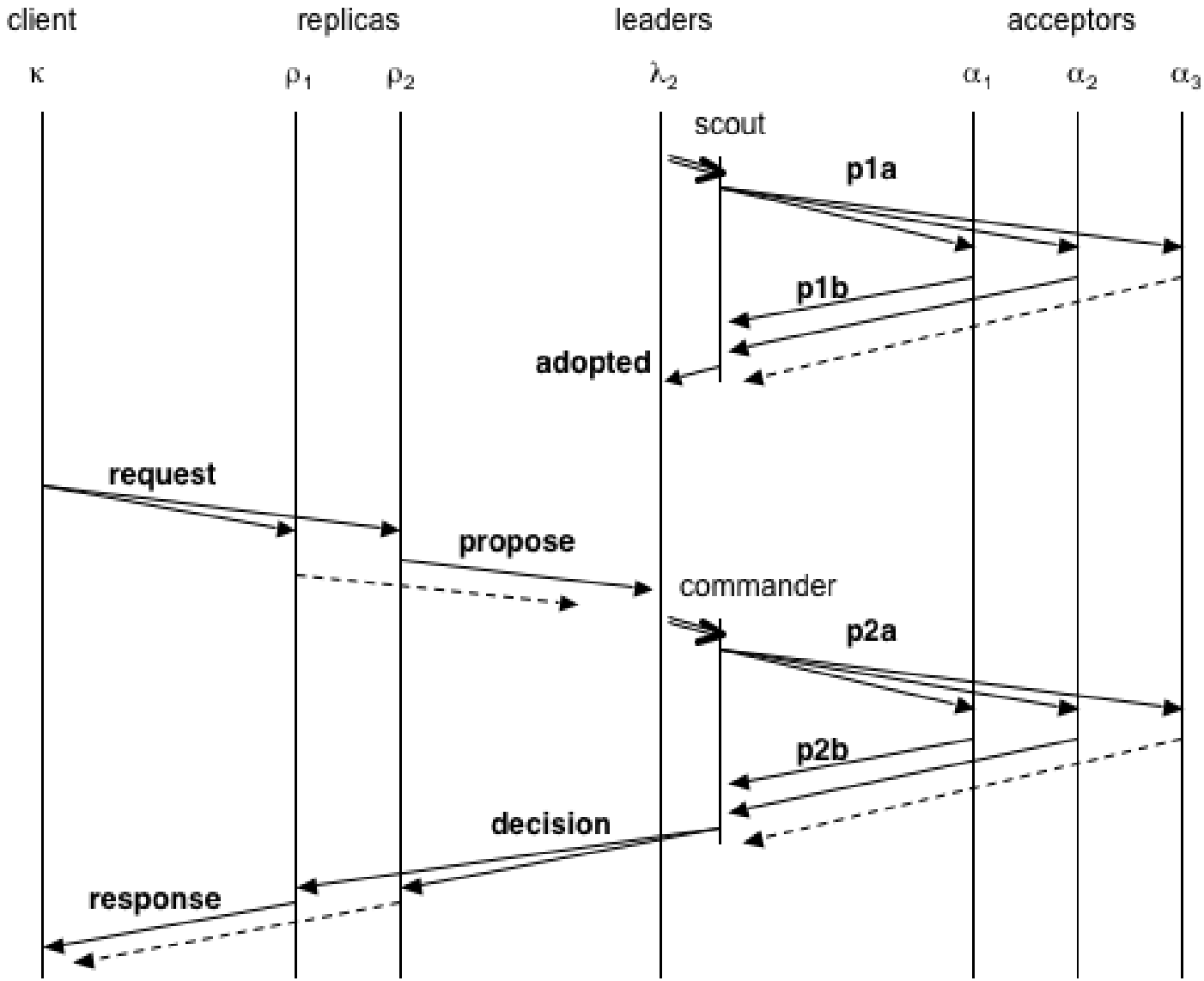
To learn that a value has been chosen, a learner must find out that a proposal has been accepted by a majority of acceptors. The obvious algorithm is to have each acceptor, whenever it accepts a proposal, respond to all learners, sending them the proposal. ...



thanks to Leslie Lamport for answering many questions!

# Multi-Paxos, van Renesse's diagram

[vR11, vRA15]



# Bitcoin protocol, picture in words

think of a chain of blocks

- that can become a tree

each miner:

- keeps the longest chain seen,
- tries to solve a hard puzzle, add the next block, gossip about it
- hopes and prays that others continue that extended chain

# Outline

Distributed consensus problems: classical, Byzantine, blockchain

Overview of well-known algorithms: ... Paxos ... Bitcoin ...

Method and language: DistAlgo, high-level, precise, executable

Exact algorithms: Paxos and Bitcoin

Demo of direct execution: only need Python installed

# Example: Two-phase commit

a coordinator and a set of cohorts try to commit a transaction,
want atomic commit—all subtransactions commit, or all abort

phase 1:

- coordinator sends a prepare to all cohorts.
- each cohort replies with a ready vote if it is prepared to commit,
  or else replies with an abort vote and aborts.

phase 2:

- if coordinator receives a ready vote from all cohorts,
  it sends a commit to all cohorts;
  each cohort commits and sends a done to coordinator;
  coordinator completes when receives a done from all cohorts.
- if coordinator receives an abort vote from any cohort,
  it sends an abort to all cohorts who sent a ready vote;
  each cohort who sent a ready vote aborts.

agreement, validity, weak termination, 4n-4 msgs



thanks to Ken Birman who gave it as challenge for LADA 2012!

# Two-phase commit in English

Phase 1: Summary of the protocol [KBL06 DB and TP]

1. The coordinator sends a *prepare message* to all cohorts.

2. Each cohort waits until it receives a *prepare message* from the coordinator. If it is prepared to commit, it forces a prepared record to its log, enters a state in which it cannot be aborted by its local control, and sends "ready" in the *vote message* to the coordinator.

If it cannot commit, it appends an abort record to its log. Or it might already have aborted. In either case, it sends "aborting" in the *vote message* to the coordinator, rolls back any changes the subtransaction has made to the database, release the subtransaction's locks, and terminates its participation in the protocol.

Phase 2:

1. The coordinator waits until it receives votes from all cohorts. If it receives at least one "aborting" vote, it decides to abort, sends an *abort message* to all cohorts that voted "ready", deallocates the transaction record in volatile memory, and terminates its participation in the protocol.

If all votes are "ready", the coordinator decides to commit (and stores that fact in the transaction record), forces a commit record (which includes a copy of the transaction record) to its log, and sends a *commit message* to each cohort.

2. Each cohort that voted "ready" waits to receive a message from the coordinator. If a cohort receives an *abort message*, it rolls back any changes the subtransaction has made to the database, appends an abort record to its log, releases the subtransaction's locks, and terminates it participation in the protocol.

If the cohort received a *commit message*, it forces a commit record to its log, releases all locks, sends a *done message* to the coordinator, and terminates its participation in the protocol.

3. If the coordinator committed the transaction, it waits until it receives *done message* from all cohorts. Then it appends a completion record to its log, deletes the transaction record from volatile memory, and terminates it participation in the protocol.

# Specifying/Programming algorithms (quickly)

significant advances in programming languages:

... ALGOL ... C++ ... Java ... Python ... Prolog ...

- statements: assignments, conditionals, loops
- expressions: arithmetic, Boolean, other data (sets)
- subroutines: functions, procedures (recursion)
- logic rules: predicates, deduction, though less used
- objects: keep data and do operations (organization)

that's mostly sequential and centralized.

# Concurrent programming (quickly)

threads: multiple threads accessing shared data

threads as concurrent objects

- the concurrent programming model in Java
- adopted by other languages, such as Python and C#

Java made concurrent programming easier.

# Distributed programming (quickly)

low-level or complex libraries, or restricted programming models

- sockets: C, Java, ... most widely used languages
- MPI: Fortran, C++, ... for high performance computing
- RPC: C, ... just about any language, Java RMI
- processes: Erlang, and more theoretically studied languages

...

study of distributed algorithms, not building real applications

- pseudocode, English: high-level but imprecise, not executable
- formal specification languages: precise but lower-level

much less progress

# Programming languages (quickly)

Erlang by Joe Armstrong etc at Ericsson
Argus by Barbara Liskov etc at MIT
Emerald from Eden by Andrew Black etc at U Washington
Lynx by Michael Scott at U Rochester
SR and MPD by Greg Andrews etc at U Arizona
Hermes and Concert/C at IBM Watson
DML on top of CML at Cornell...

Overlog by Joe Hellerstein etc at UC Berkeley
Bloom and Dedalus also by Joe Hellerstein etc
Meld by Seth Goldstein etc at CMU...

Unity, Seuss, Mozart...

libraries in C, C++, Java, Python...: socket, MPI...

# Formal specification languages (quickly)

TLA+ and PlusCal by Lamport, with TLA Toolbox
Promela by Holzmann originally at AT&T Bell Labs
Alloy by Jackson's group at MIT
Dafny by Leino at Microsoft Research and now Amazon
...

IOA and TIOA by Lynch's group at MIT
process algebra: CSP by Hoare, CCS by Milner, pi-calculus...
EventML by Constable's Nuprl group at Cornell
...

ProVerif and CryptoVerif at INRIA, Scyther at Oxford, Tamarin at ETH Zurich, AVISPA
...

# Pseudocode languages

in many textbooks and papers...

best distributed algorithm designers try to use carefully

# Languages for distributed algorithms

should combine the best of all of:

- pseudocode languages: gives high-level flow
- specification languages: has precise semantics
- programming languages: is executable

indeed should do for solving hard algorithm problems in general

DistAlgo: designed exactly to achieve this

# Example: Two-phase commit

a coordinator and a set of cohorts try to commit a transaction,
want atomic commit—all subtransactions commit, or all abort

phase 1:

- coordinator sends a prepare to all cohorts.
- each cohort replies with a ready vote if it is prepared to commit,
  or else replies with an abort vote and aborts.

phase 2:

- if coordinator receives a ready vote from all cohorts,
  it sends a commit to all cohorts;
  each cohort commits and sends a done to coordinator;
  coordinator completes when receives a done from all cohorts.
- if coordinator receives an abort vote from any cohort,
  it sends an abort to all cohorts who sent a ready vote;
  each cohort who sent a ready vote aborts.

agreement, validity, weak termination, 4n-4 msgs



thanks to Ken Birman who gave it as challenge for LADA 2012!

# How to express it

two extremes, and many in between

1. English: clear high-level flow; imprecise, informal

2. state machine based specs: precise; low-level control flow
   Nancy Lynch's I/O automata: book p183-184, but 2n-2 msgs

in between:

- Michel Raynal's pseudocode: still informal and imprecise
- Leslie Lamport's PlusCal: still complex
  (P2TwoPhase, 68 lines excluding comments and empty lines)
- Robbert van Renesse's pseudocode: precise, almost high-level

lack concepts for building real systems—much more complex
most of these are not executable at all

# Two-phase commit in English

Phase 1: Summary of the protocol [KBL06 DB and TP]

1. The coordinator sends a *prepare message* to all cohorts.

2. Each cohort waits until it receives a *prepare message* from the coordinator. If it is prepared to commit, it forces a prepared record to its log, enters a state in which it cannot be aborted by its local control, and sends "ready" in the *vote message* to the coordinator.

If it cannot commit, it appends an abort record to its log. Or it might already have aborted. In either case, it sends "aborting" in the *vote message* to the coordinator, rolls back any changes the subtransaction has made to the database, release the subtransaction's locks, and terminates its participation in the protocol.

Phase 2:

1. The coordinator waits until it receives votes from all cohorts. If it receives at least one "aborting" vote, it decides to abort, sends an *abort message* to all cohorts that voted "ready", deallocates the transaction record in volatile memory, and terminates its participation in the protocol.

If all votes are "ready", the coordinator decides to commit (and stores that fact in the transaction record), forces a commit record (which includes a copy of the transaction record) to its log, and sends a *commit message* to each cohort.

2. Each cohort that voted "ready" waits to receive a message from the coordinator. If a cohort receives an *abort message*, it rolls back any changes the subtransaction has made to the database, appends an abort record to its log, releases the subtransaction's locks, and terminates it participation in the protocol.

If the cohort received a *commit message*, it forces a commit record to its log, releases all locks, sends a *done message* to the coordinator, and terminates its participation in the protocol.

3. If the coordinator committed the transaction, it waits until it receives *done message* from all cohorts. Then it appends a completion record to its log, deletes the transaction record from volatile memory, and terminates it participation in the protocol.

# Two-phase commit in DistAlgo

```
process Coordinator:
  def setup(tid, cohorts): pass          # take in transaction id and cohorts
  def run():
    send ('prepare',tid) to cohorts
    await each c in cohorts has received ('vote',_,=tid) from c
    if each c in cohorts has received('vote','ready',=tid) from c:
      send ('commit',tid) to cohorts
      await each c in cohorts has received ('done',=tid) from c
      print(complete'+tid)
    else:
      s := {c: c in cohorts, received ('vote','ready',=tid) from c}
      send ('abort',tid) to s
    print('terminate'+tid)

process Cohort:
  def setup(f): pass                     # take in failure rate as a percentage
  def run(): await(False)
  receive ('prepare',tid) from c:
    if prepared(tid):
      send ('vote','ready,tid) to c
    else:
      send ('vote','abort',tid) to c
      abort(tid)
  receive ('commit',tid) from c:
    commit(tid)
    send ('done',tid) to c
  receive ('abort',tid):
    abort(tid)
  def prepared(tid): return randint(0,100) > f
  def abort(tid): print('abort'+tid)
  def commit(tid): print('commit'+tid)
```

solution for LADA 2012

# Complete program in DistAlgo

```
 0  from random import randint

... # content of the previous slide

31  def main():
32    cs = 25 new Cohort(10)            # create 25 cohorts with 10% failure rate
33    c = new Coordinator(0,cs)         # create 1 coordinator with trans id 0
34    cs.start()                        # start cohorts
35    c.start()                         # start coordinator
```

# DistAlgo language overview

as extensions to common object-oriented languages

including a syntax for extensions to Python

1. distributed processes and sending messages

2. control flows and receiving messages

3. high-level queries of message histories

4. configurations

# 1. Distributed processes, sending messages

process definition

```
process p: process_body                          setup, run, self
class p (process): process_body
```

process creation, setup, and start

```
v = n new p at node_exp
v = new(p, num = n, at = node_exp)
pexp.setup(args)                          new p → new p(args)
setup(pexp, (args))                  new(p) → new(p, (args))
pexp.start()
start(pexp)
```

sending messages (usually tuples)

```
send mexp to pexp
send(mexp, to = pexp)
```

# 2. Control flows, receiving messages

yield point with label

```
-- l:
-- l
```

handling messages received

```
receive mexp from pexp at l_1,...,l_j:
    handler_body
def receive(msg = mexp, from_ = pexp, at = (l_1,...,l_j)):
    handler_body
```

synchronization (nondeterminism)

```
await bexp                                    await timeout t
await(bexp)                                   await(timeout(t))
await bexp_1: stmt_1 or ... or bexp_k: stmt_k
timeout t: stmt
if await(bexp_1): stmt_1 elif ... elif bexp_k: stmt_k
elif timeout(t): stmt
```

# 3. High-level queries of message histories

message sequences: `received`, `sent`

`received` $mexp$ `from` $pexp$

$mexp$ `from` $pexp$ `in received`

`received(`$mexp$`, from_ =` $pexp$`)`

`(`$mexp$`,` $pexp$`) in received`

1) comprehensions

$\{exp$`:` $v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$`,` $bexp\}$

`setof(`$exp$`,` $v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$`,` $bexp$`)`

2) aggregates

$agg_op$ $comprehension_exp$

$agg_op$`(`$comprehension_exp$`)`

3) quantifications

`some` $v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$ `has` $bexp$

`each` $v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$ `has` $bexp$

`some(`$v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$`, has =` $bexp$`)`

`each(`$v_1$ `in` $sexp_1$`, ...,` $v_k$ `in` $sexp_k$`, has =` $bexp$`)`

tuple patterns, left side of membership clause

`_` matches anything. `=v`, `_v`: equals value of variable `v`

# 4. Configurations

channel types

```
configure channel = fifo
config(channel = fifo)                    FIFO Fifo 'fifo' ...
```

default is not FIFO or reliable

message handling

```
configure handling = all
config(handling = all)
```

default is one

logical clocks

```
configure clock = Lamport
config(clock = Lamport)
```

call `logical_time()` to get the logical time

overall: `.da` files

process definitions, method `main`, and conventional parts; `main`: configurations and process creation, setup, and start

# Optimization by incrementalization

- introduce variables to store values of queries
- transform the queries to use introduced variables
- incrementally maintain stored values at each update

**powerful**: systematic handling of

1. quantifications for synchronization as expensive queries
2. updates caused by sending, receiving, and handling of msgs in the same way as other updates in the program

transform expensive synchronization conditions into efficient tests and incremental updates as msgs are sent and received

sequences `received` and `sent` will be removed as appropriate
only values needed for incremental computation of synchronization conditions will be stored and incrementally updated

# Two-phase commit in DistAlgo

```
process Coordinator:
  def setup(tid, cohorts): pass          # take in transaction id and cohorts
  def run():
    send ('prepare',tid) to cohorts
    await each c in cohorts has received ('vote',_,=tid) from c
    if each c in cohorts has received('vote','ready',=tid) from c:
      send ('commit',tid) to cohorts
      await each c in cohorts has received ('done',=tid) from c
      print(complete'+tid)
    else:
      s := {c: c in cohorts, received ('vote','ready',=tid) from c}
      send ('abort',tid) to s
    print('terminate'+tid)

process Cohort:
  def setup(f): pass                     # take in failure rate as a percentage
  def run(): await(False)
  receive ('prepare',tid) from c:
    if prepared(tid):
      send ('vote','ready,tid) to c
    else:
      send ('vote','abort',tid) to c
      abort(tid)
  receive ('commit',tid) from c:
    commit(tid)
    send ('done',tid) to c
  receive ('abort',tid):
    abort(tid)
  def prepared(tid): return randint(0,100) > f
  def abort(tid): print('abort'+tid)
  def commit(tid): print('commit'+tid)
```

solution for LADA 2012

# Optimized after incrementalization (I)

```
process Coordinator:

  def setup(tid, cohorts):
    ncohorts := size(cohorts)          # number of cohorts
    svoted := {}                       # set of voted cohorts
    nvoted := 0                        # number of voted cohorts
    sready := {}                       # set of ready cohorts
    nready := 0                        # number of ready cohorts
    sdone := {}                        # set of done cohorts
    ndone := 0                         # number of done cohorts
  def run():
    send ('prepare',tid) to cohorts
    await nvoted = ncohorts            # replaced universal quantification
    if nready = ncohorts:              # replaced universal quantification
      send ('commit',tid) to cohorts
      await ndone = ncohorts           # replaced universal quantification
      print('complete'+tid)
    else:
      s := sready                      # replaced set query
      send ('abort',tid) to s
    print('terminate'+tid)
```

# Optimized after incrementalization (II)

```
21    receive ('vote',v,tid) from c:          # new message handler
22      if c in cohorts:
23        if c not in svoted:
24          svoted.add(c)
25          nvoted +:= 1
26        if v == 'ready':
27          if c not in sready:
28            sready.add(c)
29            nready +:= 1

30    receive ('done',tid) from c:            # new message handler
31      if c in cohorts:
32        if c not in sdone:
33          sdone.add(c)
34          ndone +:= 1

35  process Cohort:
52    ...                                     # no change

53  def main():
57    ...                                     # no change
```

# High-level executable specification of distributed algorithms

exploit high-level abstractions of computation and control

1. high-level synchronization with explicit wait on received msgs
2. high-level conditions for when to send msgs and take actions
3. high-level queries for what to send in msgs to whom
4. collective send-actions for overall computation and control

successes with many distributed algorithms

discovered improvements: for both correctness and efficiency
including for vRA Multi-Paxos, Lamport distributed mutex, use of Lamport's clock for fairness, ...
last week to the two-phase commit for LADA 2012

# Two-phase commit in DistAlgo

```
process Coordinator:
  def setup(tid, cohorts): pass          # take in transaction id and cohorts
  def run():
    send ('prepare',tid) to cohorts
    await each c in cohorts has received ('vote',_,=tid) from c
    if each c in cohorts has received('vote','ready',=tid) from c:
      send ('commit',tid) to cohorts
      await each c in cohorts has received ('done',=tid) from c
      print(complete'+tid)
    else:
      s := {c: c in cohorts, received ('vote','ready',=tid) from c}
      send ('abort',tid) to s
    print('terminate'+tid)

process Cohort:
  def setup(f): pass                     # take in failure rate as a percentage
  def run(): await(False)
  receive ('prepare',tid) from c:
    if prepared(tid):
      send ('vote','ready,tid) to c
    else:
      send ('vote','abort',tid) to c
      abort(tid)
  receive ('commit',tid) from c:
    commit(tid)
    send ('done',tid) to c
  receive ('abort',tid):
    abort(tid)
  def prepared(tid): return randint(0,100) > f
  def abort(tid): print('abort'+tid)
  def commit(tid): print('commit'+tid)
```

solution for LADA 2012

# Two-phase commit in DistAlgo

```
process Coordinator:
  def setup(tid, cohorts): pass          # take in transaction id and cohorts
  def run():
    send ('prepare',tid) to cohorts
    await each c in cohorts has received ('vote',_,=tid) from c
    if each c in cohorts has received('vote','ready',=tid) from c:
      send ('commit',tid) to cohorts
      await each c in cohorts has received ('done',=tid) from c
      print(complete'+tid)
    else:
      s := {c: c in cohorts, received ('vote','ready',=tid) from c}
      send ('abort',tid) to s
    print('terminate'+tid)

process Cohort:
  def setup(f): pass                     # take in failure rate as a percentage
  def run(): await(False)                # def run():
  receive ('prepare',tid) from c:        #   await some received ('prepare',tid) from c
    if prepared(tid):                    #   ...
      send ('vote','ready,tid) to c      #     ...
    else:                                #     await received ('commit',tid) from c
      send ('vote','abort',tid) to c     #       ...
      abort(tid)                         #       ...
  receive ('commit',tid) from c:         #     or received ('abort',tid) from c
    commit(tid)                          #       ...
    send ('done',tid) to c               #   ...
  receive ('abort',tid):                 #     ...
    abort(tid)                           #     ...
  def prepared(tid): return randint(0,100) > f
  def abort(tid): print('abort'+tid)
  def commit(tid): print('commit'+tid)
```

# DistAlgo summary

powerful, easy-to-use language: high-level, precise, executable

- distributed processes and sending messages
- yield points and handling of received messages
- await and synchronization conditions as queries of msg history
- high-level constructs for system configuration

powerful optimization: generate efficient implementations

transform expensive synchronization conditions
into efficient handlers as messages are sent and received,
by incrementalizing queries, especially logic quantifications
via incremental aggregate ops on sophisticated data structures

successes with well-known algorithms: many, simplified too

including Paxos and many variants for distributed consensus

# Outline

Distributed consensus problems: classical, Byzantine, blockchain

Overview of well-known algorithms: ... Paxos ... Bitcoin ...

Method and language: DistAlgo, high-level, precise, executable

Exact algorithms: Paxos and Bitcoin

Demo of direct execution: only need Python installed

# Exact algorithms for Paxos and Bitcoin

Basic Paxos: Lamport's English in Paxos made simple

vRA Multi-Paxos: van Renesse and Altinbuken' pseudocode and more in Paxos Made moderately complete

Bitcoin Backbrone: Garay, Kiayias and Leonardos's pseudocode in Bitcoin backbone protocol

# Basic Paxos, Lamport's description

[Lam01-SIGACT News]

Putting the actions of the proposer and acceptor together, we see that the algorithm operates in the following two phases.

**Phase 1.** (a) A proposer selects a proposal number $n$ and sends a $prepare$ request with number $n$ to a majority of acceptors.

(b) If an acceptor receives a $prepare$ request with number $n$ greater than that of any $prepare$ request to which it has already responded, then it responds to the request with a promise not to accept any more proposals numbered less than $n$ and with the highest-numbered proposal (if any) that it has accepted.

**Phase 2.** (a) If the proposer receives a response to its $prepare$ requests (numbered $n$) from a majority of acceptors, then it sends an $accept$ request to each of those acceptors for a proposal numbered $n$ with a value $v$, where $v$ is the value of the highest-numbered proposal among the responses, or is any value if the responses reported no proposals.

(b) If an acceptor receives an $accept$ request for a proposal numbered $n$, it accepts the proposal unless it has already responded to a $prepare$ request having a number greater than $n$.

...

To learn that a value has been chosen, a learner must find out that a proposal has been accepted by a majority of acceptors. The obvious algorithm is to have each acceptor, whenever it accepts a proposal, respond to all learners, sending them the proposal. ...



thanks to Leslie Lamport for answering many questions!

# Basic Paxos in DistAlgo

```
 1 process Proposer:
 2   def setup(acceptors):                          # take in set of acceptors
 3     self.majority := acceptors                   # any majority of acceptors; we use all
 4   def run():
 5     n := self                                                # Phase 1a: select proposal num n
 6     send ('prepare',n) to majority                           #  send prepare n to majority
 7     await count {a: received ('respond',=n,_) from a}        # Phase 2a: wait for response to n
 8           > (count acceptors)/2:                             #  from a majority of acceptors
 9       v := any ({v: received ('respond',=n,(n2,v)),          #  find val in accepted prop, if any
10                  n2 = max {n2: received ('respond',=n,(n2,_))} }  # having max prop num
11                or {any 1..100})                             #  or any value; use any in 1..100
12       responded := {a: received ('respond',=n,_) from a}#  find responded
13       send ('accept',n,v) to responded                     #  send accept for proposal n,v
14 process Acceptor:
15   def setup(learners): pass                    # take in set of learners
16   def run(): await false                       # wait for nothing, only to handle messages
17   receive ('prepare',n) from p:                            # Phase 1b: receive prepare n
18     if each sent ('respond',n2,_) has n > n2:              #  if n > each responded n2
19       max_prop := any {(n,v): sent ('accepted',n,v),       #  find accepted proposal, if any,
20                               n = max {n: sent ('accepted',n,_)} }  # having max prop num
21       send ('respond',n,max_prop) to p                     #  respond with n,max_prop
22   receive ('accept',n,v):                                  # Phase 2b: receive proposal n,v
23     if not some sent ('respond',n2,_) has n2 > n:          #  if not responded with larger n2
24       send ('accepted',n,v) to learners                    #  send accepted proposal n,v
25 process Learner:
26   def setup(acceptors): pass                   # take in set of acceptors
27   def run():
28     await some received ('accepted',n,v) has               # wait for some accepted proposal
29                 count {a: received ('accepted',=n,=v) from a}  # that has been accepted by
30                 > (count acceptors)/2:                     # a majority of acceptors
31       output('chosen',v)                       # output chosen value v
```

```
32 def main():
33   acceptors := 3 new Acceptor                # create 3 Acceptor processes
34   proposers := 3 new Proposer(acceptors)     # create 3 Proposer procs, pass in acceptors
35   learners := 3 new Learner(acceptors)       # create 3 Learner procs, pass in acceptors
36   acceptors.setup(learners)                  # to acceptors, pass in learners
37   (acceptors + proposers + learners).start() # start acceptors, proposers, learners
```

# Basic Paxos in DistAlgo

```
 1 process Proposer:
 2   def setup(acceptors):                          # take in set of acceptors
 3     self.majority := acceptors                   # any majority of acceptors; we use all
 4   def run():
 5     n := self                                                # Phase 1a: select proposal num n
 6     send ('prepare',n) to majority                           #  send prepare n to majority
 7     await count {a: received ('respond',=n,_) from a}        # Phase 2a: wait for response to n
 8           > (count acceptors)/2:                             #  from a majority of acceptors
 9       v := any ({v: received ('respond',=n,(n2,v)),          #  find val in accepted prop, if any
10                  n2 = max {n2: received ('respond',=n,(n2,_))} }  # having max prop num
11                or {any 1..100})                             #  or any value; use any in 1..100
12       responded := {a: received ('respond',=n,_) from a}#  find responded
13       send ('accept',n,v) to responded                     #  send accept for proposal n,v
14 process Acceptor:
15   def setup(learners): pass                    # take in set of learners
16   def run(): await false                       # wait for nothing, only to handle messages
17   receive ('prepare',n) from p:                            # Phase 1b: receive prepare n
18     if each sent ('respond',n2,_) has n > n2:              #  if n > each responded n2
19       max_prop := any {(n,v): sent ('accepted',n,v),       #  find accepted proposal, if any,
20                               n = max {n: sent ('accepted',n,_)} }  # having max prop num
21       send ('respond',n,max_prop) to p                     #  respond with n,max_prop
22   receive ('accept',n,v):                                  # Phase 2b: receive proposal n,v
23     if not some sent ('respond',n2,_) has n2 > n:          #  if not responded with larger n2
24       send ('accepted',n,v) to learners                    #  send accepted proposal n,v
25 process Learner:
26   def setup(acceptors): pass                   # take in set of acceptors
27   def run():
28     await some received ('accepted',n,v) has               # wait for some accepted proposal
29                 count {a: received ('accepted',=n,=v) from a}  # that has been accepted by
30                 > (count acceptors)/2:                     # a majority of acceptors
31       output('chosen',v)                       # output chosen value v
```

# Basic Paxos in DistAlgo (queries in red)

```
 1 process Proposer:
 2   def setup(acceptors):                          # take in set of acceptors
 3     self.majority := acceptors                   # any majority of acceptors; we use all
 4   def run():
 5     n := self                                                # Phase 1a: select proposal num n
 6     send ('prepare',n) to majority                           #  send prepare n to majority
 7     await count {a: received ('respond',=n,_) from a}        # Phase 2a: wait for response to n
 8           > (count acceptors)/2:                             #  from a majority of acceptors
 9       v := any ({v: received ('respond',=n,(n2,v)),          #  find val in accepted prop, if any
10                  n2 = max {n2: received ('respond',=n,(n2,_))} }  # having max prop num
11                or {any 1..100})                             #  or any value; use any in 1..100
12       responded := {a: received ('respond',=n,_) from a}#  find responded
13       send ('accept',n,v) to responded                     #  send accept for proposal n,v
14 process Acceptor:
15   def setup(learners): pass                    # take in set of learners
16   def run(): await false                       # wait for nothing, only to handle messages
17   receive ('prepare',n) from p:                            # Phase 1b: receive prepare n
18     if each sent ('respond',n2,_) has n > n2:              #  if n > each responded n2
19       max_prop := any {(n,v): sent ('accepted',n,v),       #  find accepted proposal, if any,
20                        n = max {n: sent ('accepted',n,_)} }  # having max prop num
21       send ('respond',n,max_prop) to p                     #  respond with n,max_prop
22   receive ('accept',n,v):                                  # Phase 2b: receive proposal n,v
23     if not some sent ('respond',n2,_) has n2 > n:          #  if not responded with larger n2
24       send ('accepted',n,v) to learners                    #  send accepted proposal n,v
25 process Learner:
26   def setup(acceptors): pass                   # take in set of acceptors
27   def run():
28     await some received ('accepted',n,v) has               # wait for some accepted proposal
29                 count {a: received ('accepted',=n,=v) from a}  # that has been accepted by
30                 > (count acceptors)/2:                     # a majority of acceptors
31       output('chosen',v)                       # output chosen value v
```

```
32 def main():
33   acceptors := 3 new Acceptor                # create 3 Acceptor processes
34   proposers := 3 new Proposer(acceptors)     # create 3 Proposer procs, pass in acceptors
35   learners := 3 new Learner(acceptors)       # create 3 Learner procs, pass in acceptors
36   acceptors.setup(learners)                  # to acceptors, pass in learners
37   (acceptors + proposers + learners).start() # start acceptors, proposers, learners
```

# Understanding Basic Paxos

High-level ideas: think 2-Phase commit, use majority not all, and do rounds with increasing proposal numbers.

1a: straightforwardly follows the English.
general: next larger proposal number. will see in Multi-Paxos

1b: when an acceptor receives a `prepare` with a number $>$ all numbers in its previous responses, it `responds` with this number and with any sent accepted `(n,v)` pair where `n` is max among all pairs it has sent in `accepted`;
note: if it has not sent any `accepted`, `max_prop` is `undefined`.

2a: when a proposer receives responses to its `n` from majority of acceptors, it takes `v` in the `(n2,v)` that has max `n2` in all responses to `n`, or any value in 1..100 if responses contain no `(n2,v)` pairs but only `undefined`;
note: in latter case, the set before 1..100 is empty because `undefined` does not match `(n2,v)`.
proposer then sends `accept n v` to acceptors responded to `n`.

2b: directly follows the English.

**key**: if any n, majority accepted n v, then larger n pairs with v

# Fundamental in distributed algorithms

leader election: do in 1a, then can start 2a

majority or quroum voting: end 1a and learn

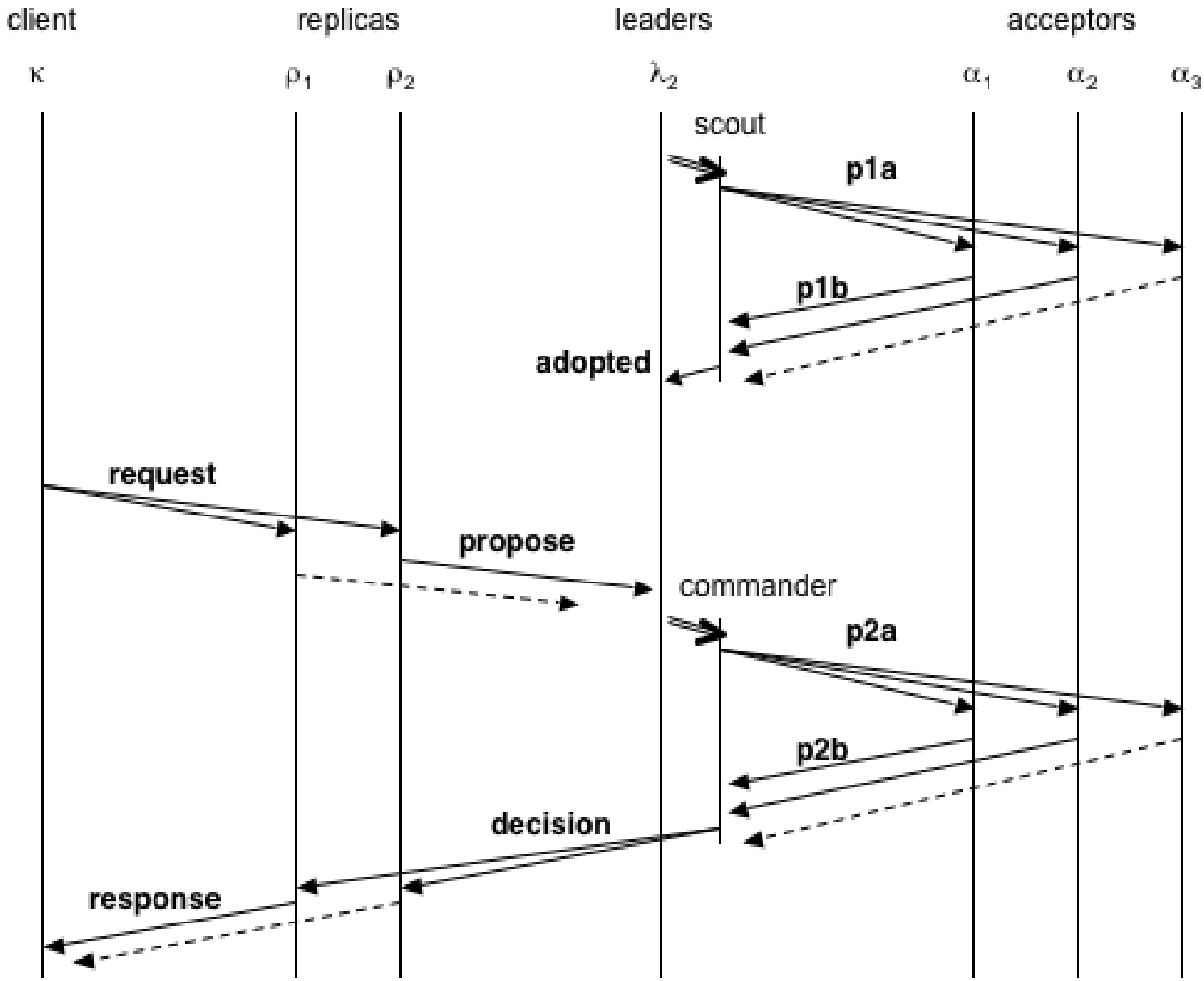
successive rounds: use increasingly larger numbers in 1a;
will see more in Multi-Paxos

timeout and failure detection: need timeout for await
to end 1a and learn; need more; will see in Multi-Paxos

selection with maximum or minimum: break symmetry,
and/or to make monotonic progress

# Multi-Paxos, van Renesse's diagram

[vR11, vRA15]



# vRA Multi-Paxos pseudocode

agree on a sequence of values:
more and better than running Basic Paxos for each slot

preemption: allows a leader to be preempted by another leader that has a larger ballot,
i.e., if a leader receives a message with a larger ballot than its own, it abandons its own and uses a larger ballot later.

replicated state machines:
replicas each keep the state of the application

reconfiguration: allows switching to a set of new leaders during execution of the algorithm.
the slot for changing the leaders must be agreed on by the old leaders, by taking the change as a command to be agreed on

issues and fixes

optimizations and executions: state reduction, failure detection

# vRA Multi-Paxos terminology

except for Acceptor, all other names used are either changed:

| Algorithm | Process Types | Data Types | | Message Types | | | |
|---|---|---|---|---|---|---|---|
| Basic Paxos | Proposer, Learner | proposal number | value | `prepare` | `respond` | `accept` | `accepted` |
| vRA Multi-Paxos | Leader, Scout, Commander | ballot number | com-mand | `1a` | `1b` | `2a` | `2b` |

or new:

| Algorithm | Process Types | Data Types | Message Types | | | | |
|---|---|---|---|---|---|---|---|
| vRA Multi-Paxos | Replica | slot | `request` | `response` | `propose` | `decision` | `preempt` |

# vRA Multi-Paxos pseudocode (I)

## Replica processes

```
process Replica(leaders, initial_state)
  var state := initial_state, slot_in := 1, slot_out := 1;
  var requests := ∅, proposals := ∅, decisions := ∅
  function propose()
    while slot_in < slot_out + WINDOW ∧ ∃c : c ∈ requests do
      if ∃op : ⟨slot_in − WINDOW, ⟨·, ·, op⟩⟩ ∈ decisions ∧ isreconfig(op) then
        leaders := op.leaders;
      end if
      if ∄c′ : ⟨slot_in, c′⟩ ∈ decisions then
        requests := requests \ {c};
        proposals := proposals ∪ {⟨slot_in, c⟩};
        ∀λ ∈ leaders : send(λ, ⟨propose, slot_in, c⟩);
      end if
      slot_in := slot_in + 1;
    end while
  end function
  function perform(⟨κ, cid, op⟩)
    if (∃s : s < slot_out ∧ ⟨s, ⟨κ, cid, op⟩⟩ ∈ decisions) ∨ isreconfig(op) then
      slot_out := slot_out + 1;
    else
      ⟨next, result⟩ := op(state);
      atomic
        state := next; slot_out := slot_out + 1;
      end atomic
      send(κ, ⟨response, cid, result⟩);
    end if
  end function
  for ever
    switch receive()
      case ⟨request, c⟩ :
        requests := requests ∪ {c″};
      end case
      case ⟨decision, s, c⟩ :
        decisions := decisions ∪ {⟨s, c⟩};
        while ∃c′ : ⟨slot_out, c′⟩ ∈ decisions do
          if ∃c″ : ⟨slot_out, c″⟩ ∈ proposals then
            proposals := proposals \ {⟨slot_out, c″⟩};
            if c″ ≠ c′ then
              requests := requests ∪ {c″};
            end if
          end if
          perform(c');
        end while
      end case
    end switch
    propose();
  end for
end process
```

# vRA Multi-Paxos pseudocode (II)

Scout and Commander: spawned by Leader for each ballot

```
process Scout(λ, acceptors, b)
  var waitfor := acceptors, pvalues := ∅;
  ∀α ∈ acceptors : send(α, ⟨p1a, self(), b⟩);
  for ever
    switch receive()
      case ⟨p1b, α, b′, r⟩ :
        if b′ = b then
          pvalues := pvalues ∪ r;
          waitfor := waitfor \ {α};
          if |waitfor| < |acceptors|/2 then
            send(λ, ⟨adopted, b, pvalues⟩);
            exit();
          end if
        else
          send(λ, ⟨preempted, b′⟩);
          exit();
        end if
      end case
    end switch
  end for
end process
```

```
process Commander(λ, acceptors, replicas, ⟨b, s, c⟩)
  var waitfor := acceptors;
  ∀α ∈ acceptors : send(α, ⟨p2a, self(), ⟨b, s, c⟩⟩);
  for ever
    switch receive()
      case ⟨p2b, α, b′⟩ :
        if b′ = b then
          waitfor := waitfor \ {α};
          if |waitfor| < |acceptors|/2 then
            ∀ρ ∈ replicas :
              send(ρ, ⟨decision, s, c⟩);
            exit();
          end if
        else
          send(λ, ⟨preempted, b′⟩);
          exit();
        end if
      end case
    end switch
  end for
end process
```

# vRA Multi-Paxos pseudocode (III)

Leader

```
process Leader(acceptors, replicas)
  var ballot_num := (0, self()), active := false, proposals := ∅;
  spawn(Scout(self(), acceptors, ballot_num));
  for ever
    switch receive()
      case ⟨propose, s, c⟩ :
        if ∄c' : ⟨s, c'⟩ ∈ proposals then
          proposals := proposals ∪ {⟨s, c⟩};
          if active then
            spawn(Commander(self(), acceptors, replicas, ⟨ballot_num, s, c⟩));
          end if
        end if
      end case
      case ⟨adopted, ballot_num, pvals⟩ :
        proposals := proposals ◁ pmax(pvals);
        ∀⟨s, c⟩ ∈ proposals :
          spawn(Commander(self(), acceptors, replicas, ⟨ballot_num, s, c⟩));
        active := true;
      end case
      case ⟨preempted, ⟨r', λ'⟩⟩ :
        if (r', λ') > ballot_num then
          active := false;
          ballot_num := (r' + 1, self());
          spawn(Scout(self(), acceptors, ballot_num));
        end if
      end case
    end switch
  end for
end process
```

# vRA Multi-Paxos pseudocode (IV)

$$pmax\langle pvals\rangle \equiv \{\langle s,c\rangle | \exists b : \langle b,s,c\rangle \in pvals \\ \wedge \forall b', c' : \langle b', s, c'\rangle \in pvals \Rightarrow b' \leq b\}$$

$$x \triangleleft y \equiv \{\langle s,c\rangle | \langle s,c\rangle \in y \\ \vee (\langle s,c\rangle \in x \wedge \nexists c' : \langle s,c'\rangle \in y)\}$$

Acceptor

```
process Acceptor()
  var ballot_num := ⊥, accepted := ∅;
  for ever
    switch receive()
      case ⟨p1a, λ, b⟩ :
        if b > ballot_num then
          ballot_num := b;
        end if
        send(λ, ⟨p1b, self(), ballot_num, accepted⟩);
      end case
      case ⟨p2a, λ, ⟨b, s, c⟩⟩ :
        if b = ballot_num then
          accepted := accepted ∪ {⟨b, s, c⟩};
        end if
        send(λ, ⟨p2b, self(), ballot_num⟩);
      end case
    end switch
  end for
end process
```

# Simplifications from vRA Multi-Paxos pseudocode

removed Scout and Commander:

roles for 2 phases → Leader; roles for preemption → Acceptor

replaced repeated and scattered updates by high-level queries:

in all processes: many

a simplest example: next slide

declarative, i.e., no updates of state,

except for: ballot in Leader, and application state in Replica

# Example: Commander in vRA Multi-Paxos

```
process Commander:
  def setup(leader, acceptors, replicas, b,s,c): pass
  def run():
    send ('2a', b,s,c) to acceptors
    await count {a: received(('2b', =b) from a)}
          > (count acceptors)/2:
       send ('decision', s,c) to replicas
    or some received('2b', b2) has b2 != b:
       send ('preempted', b2) to leader
```

clear synchronization condition:
high-level queries, no updates

an exercise:
find a bug on in box on right if
right side of $<$ uses integer division

process *Commander*($\lambda$, *acceptors*, *replicas*, $\langle b, s, c\rangle$)
  var *waitfor* := *acceptors*;
  $\forall \alpha \in$ *acceptors* : *send*($\alpha, \langle$**p2a**, *self*(), $\langle b, s, c\rangle\rangle$);
  for ever
    switch *receive*()
      case $\langle$**p2b**, $\alpha, b'\rangle$ :
        if $b' = b$ then
          *waitfor* := *waitfor* $\setminus \{\alpha\}$;
          if $|$*waitfor*$| < |$*acceptors*$|/2$ then
            $\forall \rho \in$ *replicas* :
              *send*($\rho, \langle$**decision**, $s, c\rangle$);
            *exit*();
          end if
        else
          *send*($\lambda, \langle$**preempted**, $b'\rangle$);
          *exit*();
        end if
      end case
    end switch
  end for
end process

# Resulting vRA Multi-Paxos in DistAlgo

```
 1 process Replica:
 2   def setup(leaders, state): pass                          # take in initial set of leaders and state
 3   def run():
 4     slot_in, slot_out := 1, 1                              # slot to send prop, slot to apply decision
 5     while true:
 6       await slot_in < slot_out + WINDOW and                        # if slot_in can be increased and
 7             some received ('request',c) has                        # some received request for cmd c
 8               each sent ('propose',s,=c) has                       # each sent proposed slot s for c
 9                 some received ('decision',=s,c2) has c2 != c:       # some rcvd dec has s for diff c2
10         if some received ('decision', slot_in - WINDOW, (_,_,op)) has is_reconfig(op):
11           leaders := op.leaders                            # if slot_in-WINDOW is reconfig, set leaders
12         if not some received ('decision',=slot_in,_): # if slot_in is not decided
13           send ('propose', slot_in, c) to leaders        #   propose slot_in for command c
14         slot_in := slot_in + 1
15       or some received ('decision', =slot_out, c):               # if rcvd dec slot_out for some c
16         client, cmd_id, op := c                          # extract components of command c
17         if not (some received ('decision',s,=c) has s < slot_out) and not is_reconfig(op):
18           state, result := apply(op, state)              # if c not decided before & is not reconfig
19           send ('response', cmd_id, result) to client #   apply op and send result to client
20         slot_out := slot_out + 1
```

```
21 process Leader:
22   def setup(acceptors, replicas): pass                  # take in sets of acceptors and replicas
23   def run():
24     ballot := (0, self)                                 # ballot num is pair of round num and self
25     while true:
26       send ('1a', ballot) to acceptors                  # send 1a with ballot number           # 1a
27       await count {a: received ('1b',=ballot,_) from a} > (count acceptors)/2:              # 2a
28         ps := {p: received ('1b',=ballot,accepted), p in accepted}  # all proposals accepted #
29         for (s,c) in {(s,c): (b,s,c) in ps, b = max {b2: (b2,=s,_) in ps}}:  # max b per s   #
30           send ('2a', ballot, s, c) to acceptors       # send 2a for previously accepted s,c  #
31         while true:                                                                          #
32           await some received ('propose',s,c) has not some sent ('2a',=ballot,=s,_):        #
33             send ('2a', ballot, s, c) to acceptors     # send 2a for newly proposed s,c       #
34           or some received ('2b',=ballot,s,c) has                                  # learn
35                count {a: received ('2b',=ballot,=s,=c) from a} > (count acceptors)/2:  #
36             send ('decision', s, c) to replicas        # send decided s,c to replicas    #
37           or some received ('preempt',(r2,leader2)) has (r2,leader2) > ballot: break  # preempted
38       or some received ('preempt',(r2,leader2)) has (r2,leader2) > ballot: pass       # preempted
39       ballot := (r2+1, self)                            # increment round number in ballot number
40 process Acceptor:
41   def run(): await false                                # wait for nothing, only to handle messages
42   receive ('1a', b) from leader:                        # Phase 1b: receive 1a with ballot b
43     if each sent ('1b',b2,_) has b > b2:                #  if b > each b2 in sent 1b msgs
44       accepted := {(b,s,c): sent ('2b',b,s,c)}          #  get accepted triples sent in 2b msgs
45       send ('1b', b, accepted) to leader                #  send 1b with b and accepted triples
46   receive ('2a', b, s, c) from leader:                  # Phase 2b: receive 2a with bal,slot,cmd
47     if not some sent ('1b',b2,_) has b2 > b:            #  if not sent 1b with larger ballot b2
48       send ('2b', b, s, c) to leader                    #  send 2b to accept the proposal triple
49   receive m from leader:                                # Preemption: receive 1a or 2a msg
50     maxb := max ({b: received ('1a',b)} + {b: received ('2a',b,_,_)})  # find max bal in 1a,2a msgs
51     if m[1] < maxb: send ('preempt', maxb) to leader  # if ballot in m < max bal, send preempt
```

# Basic Paxos in DistAlgo

```
 1 process Proposer:
 2   def setup(acceptors):                          # take in set of acceptors
 3     self.majority := acceptors                   # any majority of acceptors; we use all
 4   def run():
 5     n := self                                                # Phase 1a: select proposal num n
 6     send ('prepare',n) to majority                           #  send prepare n to majority
 7     await count {a: received ('respond',=n,_) from a}        # Phase 2a: wait for response to n
 8           > (count acceptors)/2:                             #  from a majority of acceptors
 9       v := any ({v: received ('respond',=n,(n2,v)),          #  find val in accepted prop, if any
10                  n2 = max {n2: received ('respond',=n,(n2,_))} }  # having max prop num
11                or {any 1..100})                             #  or any value; use any in 1..100
12       responded := {a: received ('respond',=n,_) from a}#  find responded
13       send ('accept',n,v) to responded                     #  send accept for proposal n,v
14 process Acceptor:
15   def setup(learners): pass                    # take in set of learners
16   def run(): await false                       # wait for nothing, only to handle messages
17   receive ('prepare',n) from p:                            # Phase 1b: receive prepare n
18     if each sent ('respond',n2,_) has n > n2:              #  if n > each responded n2
19       max_prop := any {(n,v): sent ('accepted',n,v),       #  find accepted proposal, if any,
20                               n = max {n: sent ('accepted',n,_)} }  # having max prop num
21       send ('respond',n,max_prop) to p                     #  respond with n,max_prop
22   receive ('accept',n,v):                                  # Phase 2b: receive proposal n,v
23     if not some sent ('respond',n2,_) has n2 > n:          #  if not responded with larger n2
24       send ('accepted',n,v) to learners                    #  send accepted proposal n,v
25 process Learner:
26   def setup(acceptors): pass                   # take in set of acceptors
27   def run():
28     await some received ('accepted',n,v) has               # wait for some accepted proposal
29                 count {a: received ('accepted',=n,=v) from a}  # that has been accepted by
30                 > (count acceptors)/2:                     # a majority of acceptors
31       output('chosen',v)                       # output chosen value v
```

# Issues and fixes for vRA Multi-Paxos

small inefficiency (**but violating invariants**):
was hard to undertand, find invariants, and write queries

- useless replies in Acceptor are fixed
- unnecessary delays in Replica are fixed:

liveness violations (**if messages can be lost**):
was easy to discover after high-level queries are used

- liveness violation in Replica and fix
- liveness violation in Leader and fix

paper uses fair links assumption, but that's not practical
(they later tried to solve that but had other problems)

# Optimization: State reduction

the most serious efficiency problem: fast growing set `accepted` in `1b` msgs, choking execution that does real message passing

solution: not keep all triples in sent `2b` messages,
but keep only triples with max ballot for each slot,
so at most one triple for each slot

done by changing line 44 to

```
44  accepted := {(b,s,c): sent ('2b',b,s,c),
                          b = max {b: sent ('2b',b,=s,_)}}
```

reduces not only size of `1b` messages, but also space needed by acceptors and leaders, which send and receive `1b` messages

# Optimization: Failure detection

the next most serious problem:

leaders compete to become the leader with the max ballot, leaving little or no time for proposals to be decided

solution: add failure detection using ping-pong once preempted:

before incrementing `ballot`, periodically ping `leader2` who has the max ballot `(r2,leader2)` and wait for pong

done by inserting

```
38.1   while each sent('ping',=r2,t) to =leader2
                has received ('pong',r2,t) from leader2:
38.2     send ('ping', r2, logical_time()) to leader2
38.3     await timeout TIMEOUT
```

and adding `receive` definition

```
39.1   receive ('ping', r2, t) from leader2:
39.2     send ('pong', r2, t) to leader2
```

# Bitcoin blockchain consensus

The Bitcoin Backbone Protocol: Analysis and Applications
16 versions since Sep 2014, last version July 1, 2019.
https://eprint.iacr.org/eprint-bin/versions.pl?entry=2014/765

C: chain of blocks, each of form (s, x, ctr)
s: hash of previous block
x: content of this block, i.e., transactions in this block
ctr: number of times hash was attempted for creating block

# Bitcoin backbone in DistAlgo

## hash and block functions:

```
 1 q = 2**32  # max allowed number of hash queries in a round, to bound ctr

 2 T = 2**241-1  # "difficulty" of POW, chance of 1/(2**15) for hash 0..2**256-1

 3 def G(s, x):  # SHA-256 hash of s--hash H of previous block, and x--input transaction
 4   return int(hashlib.sha256((str(s) + str(x)).encode()).hexdigest(),base=16)

 5 def H(ctr, g):  # SHA-256 hash of ctr--number of times H was tried, and g--hash from G
 6   return int(hashlib.sha256((str(ctr) + str(g)).encode()).hexdigest(),base=16)

 7 def validblock(s, x, ctr):  # hash satisfies difficulty and ctr is bounded by q
 8   return H(ctr, G(s, x)) < T and ctr <= q



 9 def V(C):  # content validation predicate  # whether contents in blocks in chain C
10   def proper(x): return True               # are proper for the application
11   return each((_, x, _) in C, has= proper(x))

   # def I(st, C, round, INPUT, RECEIVE): pass  # input contribution function
   #   # simply copying contents of input tape into x and keeping st = epsilon.

12 def R(C):  # chain reading function  # list of contents x in blocks of C
13   return [x for (_, x, _) in C]
```

# Bitcoin backbone in DistAlgo

## proof-of-work blockchain protocol:

```
14 def validate(C):  # whether V(C), each block is valid, and hash of block = 1st of next block
15   return (V(C) and each((s,x,ctr) in C, has= validblock(s,x,ctr)) and\
16           each((i,(s,x,ctr)) in enumerate(C[:-1]), has= H(ctr,G(s,x))==C[i+1][0]))

17 def pow(x, C):    # new block containing content x if mined successfully, None otherwise
18   if not C: s = 0
19   else:
20     s2, x2, ctr2 = C[-1]  # last block
21     s = H(ctr2, G(s2, x2))
22   g = G(s, x)
23   for ctr in range(1,q+1):
24     if H(ctr, g) < T:
25       return (s, x, ctr)

26 class Miner(process): # Bitcoin backbone protocol
27   def setup(miners):  # take in set of miners
28     self.C = []       # a longest chain seen
29   def run(): await False
30   def receive(msg=('CHAIN', C2)):
31     if validate(C2) and len(C2) > len(C): C = C2
32   def receive(msg='READ'):
33     output('OUTPUT:', R(C))
34   def receive(msg=('INSERT', x)):
35     B = pow(x, C)
36     if B:
37       C.append(B)
38       send(('CHAIN', C), to= miners-{self})
```

# Bitcoin backbone in DistAlgo

## test environment and overall execution:

```
0  import hashlib, random

   ...

39 class Environment(process):
40   def setup(miners, num_trans): pass
41   def run():
42     trans = random.sample(range(100,1000), num_trans)  # input transactions
43     output('====', trans)
44     for x in trans:                                    # for each transaction
45       target_miners =  random.sample(miners, 2)        # pick 2 miners
46       output('= = = = =', x, target_miners)
47       send(('INSERT', x), to= target_miners)           # send INSERT transaction
48       time.sleep(.3)
49     time.sleep(3)
50     send('READ', to= miners)                           # send READ chain

51 def main():
52   n = 4                                                # number of miners
53   num_trans = 4                                        # number of transactions

54   miners = new(Miner, num= n)                          # create miners
55   for m in miners: setup(m, [miners])                  # set up miners
56   env = new(Environment, (miners, num_trans))          # create, set up env
57   start(miners | {env})                                # start miners, env
```

# Improvements from pseudocode in paper

the resulting 1-slide proof-of-work blockchain protocol shows:

- queries $\leftarrow$ precise code $\leftarrow$ pseudocode
- simpler code $\leftarrow$ precise code $\leftarrow$ pseudocode
- better protocol $\leftarrow$ precise code $\leftarrow$ pseudocode

all from precise code

# Queries ← precise code ← pseudocode

```
def validate(C):  # whether V(C), each block is valid, and hash of block = 1st of next block
  return (V(C) and each((s,x,ctr) in C, has= validblock(s,x,ctr)) and \
          each((i,(s,x,ctr)) in enumerate(C[:-1]), has= H(ctr,G(s,x))==C[i+1][0]))
```

```
# c = C.copy()
# b = V(c)
# if b and len(c)>0:
#    s, x, ctr = c[-1]
#    s2 = H(ctr,G(s,x))
#    while len(c) > 0:
#       s, x, ctr = c[-1]
#       if validblock(s,x,ctr) and H(ctr,G(s,x))==s2:
#          s2 = s
#          if len(c)>0: del c[-1]
#       else:
#          b = False
#          break
# return b
```

```
# b <- V(xC)
# if b /\ (C != epsilon) then
#   <s,x,ctr> <- head(C)
#   s' = H(ctr,G(s,x))
#   repeat
#     <s,x,ctr> <- head(C)
#     if validblock(s,x,ctr) /\ (H(ctr,G(s,x))=s') then
#       s' <- s <|Retain the Hash value
#       C <- tail(C)
#     else
#       b <- False
#   until (C = epsilon) V (b = False)
# return b
```

```
def maxvalid(Cs):  # longest valid chain in Cs # this will be dead and removed
  valid = [C for C in Cs if validate(C)]
  return C if some(C in valid, has=len(C)==max(len(C) for C in valid)) else []
```

```
# temp = []
# for Ci in Cs:
#    if validate(Ci):
#       if len(Ci) > len(temp): temp = Ci
# return temp
```

```
# temp <- epsilon
# for i = 1 to k do
#   if validate(Ci) then
#     temp <- max(Ci, temp)
# return temp
```

# Simpler code ← precise code ← pseudocode

```
def pow(x, C):  # extended chain by new block containing content x if mined successfully or None
# c = C.copy()
  if not C:                                  # if C = epsilon then
     s = 0                                   #   s <- 0
  else:                                      # else
     s2, x2, ctr2 = C[-1]  # last block      #   <s',x',ctr'> <- head(C)
     s = H(ctr2, G(s2, x2))                  #   s <- H(ctr', G(s', x'))
# ctr = 1                                    # ctr <- 1
                                             # B <- epsilon
  h = G(s, x)                                # h <- G(s,x)
# while ctr <= q:                            # while (ctr <= q)
  for ctr in range(1,q+1):                   #
     if H(ctr, h) < T:                       #   if (H(ctr, h) < T) then
        B = (s, x, ctr)                      #     B <- <s,x,ctr>
#       break                                #     break
        return B
#    ctr = ctr + 1                           #   ctr <- ctr + 1
# if B: c.append(B)                          # C <- C B
# return c                                   # return C
```

# Better protocol ← precise code ← pseudocode

```
class Miner(process, Output):                    # Bitcoin backbone protocol
  def setup(miners):  # take in all miners
    self.C = []       # a longest chain seen

  def run(): await False                         # if (init) then
                                                 #   C <- epsilon
                                                 #   st <- epsilon
                                                 #   round <- 1
                                                 #   init <- False
                                                 # else
  def receive(msg=('CHAIN', C2), from_=p):       #   C~ <- maxvalid(C, any chain C' in RECEIVE())
    if validate(C2) and len(C2) > len(C): C = C2
  def receive(msg='READ', from_=p):              #   if INPUT() contains READ then
    output('OUTPUT:', R(C))                      #     write R(C~) to OUTPUT
  def receive(msg=('INSERT', x), from_=p):       #   <st, x> <- I(st,C~,round,INPUT(),RECEIVE())
    B = pow(x, C)                                #   Cnew <- pow(x, C~)
    if B:                                        #   if C != Cnew then
      C.append(B)                                #     C <- Cnew
      send(('CHAIN', C), to= miners-{self})      #     Diffuse(C)
                                                 #   else
                                                 #     Diffuse(null)
                                                 #   round <- round + 1
```

# Better protocol ← precise code ← pseudocode

```
class Miner(process, Output):                  # Bitcoin backbone protocol
  def setup(miners):  # take in all miners
    self.msgs_new = []
  def receive(msg=m, from_= p):
    msgs_new.append(m)
  def run():                                   # if (init) then
    C = []             # a longest chain seen#   C <- epsilon
                                               #   st <- epsilon
                                               #   round <- 1
    while True:                                #   init <- False
      await msgs_new                           # else
      msgs = msgs_new; msgs_new = []
      C = maxvalid([C] + listof(C2, ('CHAIN', C2) in msgs))
                                               #   C~ <- maxvalid(C, any chain C' in RECEIVE())
      if ('READ', None) in msgs:               #   if INPUT() contains READ then
        output('OUTPUT':, R(C))                #     write R(C~) to OUTPUT
      if some(('INSERT', x) in msgs):          #   <st, x> <- I(st,C~,round,INPUT(),RECEIVE())
#       Cnew = pow(x, C)                       #   Cnew <- pow(x, C~)
        B = pow(x, C)                          #
#       if C != Cnew:                          #   if C != Cnew then
        if B:                                  #
#         C = Cnew                             #     C <- Cnew
          C.append(B)                          #
          send(('CHAIN', C), to= miners-{self})#  Diffuse(C)
                                               #   else
                                               #     Diffuse(null)
                                               #   round <- round + 1
```

# Outline

Distributed consensus problems: classical, Byzantine, blockchain

Overview of well-known algorithms: ... Paxos ... Bitcoin ...

Method and language: DistAlgo, high-level, precise, executable

Exact algorithms: Paxos and Bitcoin

Demo of direct execution: only need Python installed

# See DistAlgo run, and run it yourself!

get and run DistAlgo:

extremely easy, thanks to fabulous Bo Lin, and Python!

demo:

Basic Paxos, vRA Multi-Paxos, Bitcoin Backbone, ...
anything exciting you have if there is time or afterward
much more unofficial by students: failure injection, backup-restore, testing, debugging, visualization, ...

experimented with many well-known algorithms:

about all that we and hundreds of students found interesting

were helped significantly in teaching and research:

100+ different course projects at Stony Brook (incl. undergrads; and high school students who had done little or no programming).
external too: Jennifer Welch, Stephan Merz courses Spring 2019
so few bug found (none by external Spring 2019 users)

# Get and run DistAlgo

`http://github.com/DistAlgo` README

- can just download — unzip — run script without installation
- or, to install: add to python path or run `python setup.py install`
- or, not even download if you use pip: run `pip install pyDistAlgo`

  to use Python 3.7, run `pip install --pre pyDistAlgo`

`http://distalgo.cs.stonybrook.edu`

- tutorial (to update)
- language description
- formal operational semantics

# Demo

# Implementations of Lamport's dist. mutex

| Language | Dist. programming features used | Total | Clean |
|---|---|---|---|
| C | TCP socket library | 358 | 272 |
| Java | TCP socket library | 281 | 216 |
| Python | multiprocessing package | 165 | 122 |
| Erlang | built-in message passing | 177 | 99 |
| PlusCal | single process simulation with array | 134 | 90 |
| DistAlgo | built-in high-level synchronization | 48 | 32 |

program size in total number of lines of code,
and number of lines excluding comments and empty lines

# Program size for well-known algorithms

| Algorithm | DistAlgo | PlusCal | IOA | Overlog | Bloom |
|---|---|---|---|---|---|
| La mutex | 32 | 90 | 64 | | |
| La mutex2 | 33 | | | | |
| RA mutex | 35 | | | | |
| RA token | 43 | | | | |
| SK token | 42 | | | | |
| CR leader | 30 | | 41 | | |
| HS leader | 56 | | | | |
| 2P commit | 44 | 68 | | | 85 |
| DS crash | 22 | | | | |
| La Paxos | 43 | 83 | 145 | 230 | 157 |
| CL Paxos | 63 | 166 | | | |
| vR Paxos | 160 | | | | |

number of lines excluding comments and empty lines,
compared with specifications written by others in other languages

Spring 2012 [OOPSLA 2012]

# Compilation time and generated prog. sizes

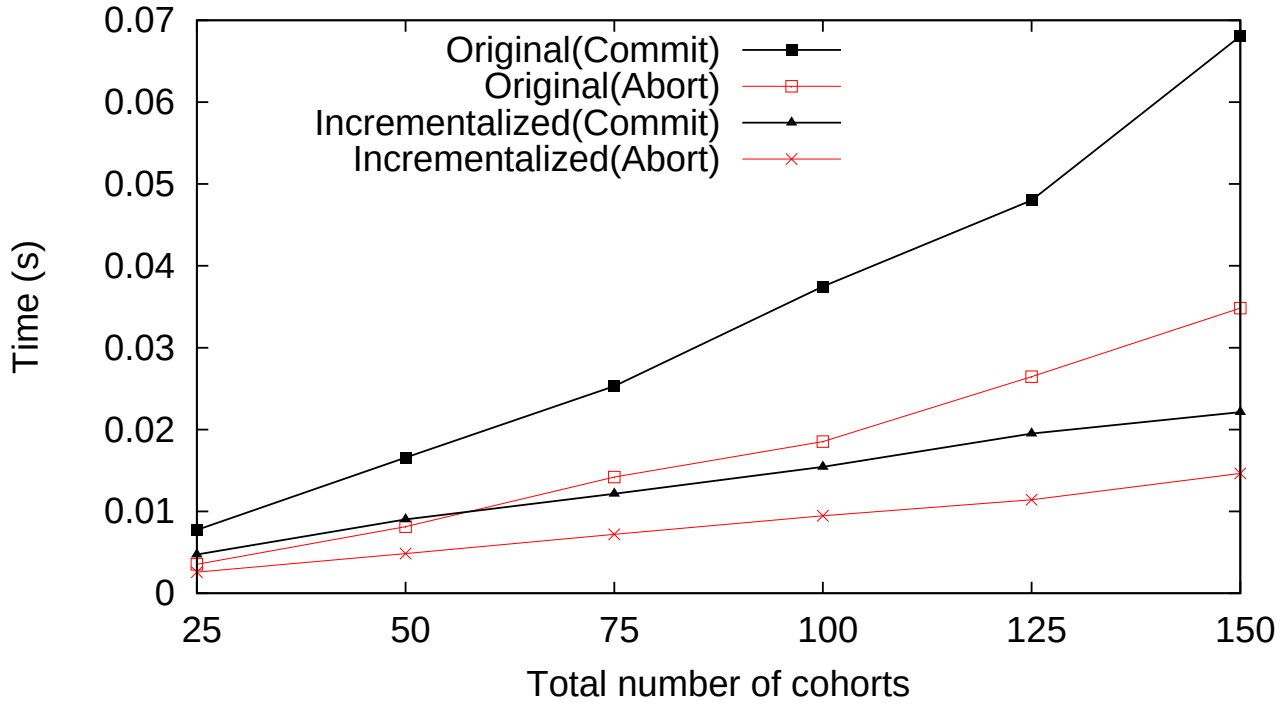
| Algorithm | Compilation time (ms) | DistAlgo size | Compiled size | Incremental-ized size |
|---|---|---|---|---|
| La mutex | 13.3 | 32 | 1395 | 1424 |
| La mutex2 | 15.3 | 33 | 1402 | 1433 |
| RA mutex | 12.3 | 35 | 1395 | 1395 |
| RA token | 12.9 | 43 | 1402 | 1402 |
| SK token | 16.5 | 42 | 1405 | 1407 |
| CR leader | 10.7 | 30 | 1395 | 1395 |
| HS leader | 18.7 | 56 | 1415 | 1415 |
| 2P commit | 21.4 | 44 | 1432 | 1437 |
| DS crash | 10.5 | 22 | 1399 | 1414 |
| La Paxos | 20.7 | 43 | 1428 | 1498 |
| CL Paxos | 32.3 | 63 | 1480 | 1530 |
| vR Paxos | 43.4 | 160 | 1555 | 1606 |

compilation time not including incrementalization time (all $<$ 30s), and numbers of lines excluding comments and empty lines of generated programs (including 1300 lines of fixed library code)

Spring 2012 [OOPSLA 2012]

# Performance of generated implementation

running time for two-phase commit in commit and abort cases:




CPU time, for failure rates of 0 (Commit) and 100 (Abort), averaged over 50 rounds and 15 independent runs.

Spring 2012 [LADA 2012]

# Example grad/undergrad projects in DistAlgo

| Algorithm | Description |
|---|---|
| Paxos | fast; vertical; Byzantine |
| Chubby | Google's distributed lock service |
| ZooKeeper | Apache distributed coordination service |
| Chord | distributed hash table (DHT) |
| Tapestry | DHT |
| Pastry | DHT |
| Kademlia | DHT, used in BitTorrent |
| Dynamo | Amazon's distributed key-value store |
| GFS | Google file system |
| HDFS | Hadoop distributed file system |
| Bigtable | Google's distributed storage for structured data |
| Cassandra | Apache distributed database |
| Megastore | Google's transactional storage for interactive services |
| MapReduce | simplified data processing on large clusters |
| AODV | wireless mesh network routing |
| NS key | Needham Shroeder public/shared key protocols |

each is about 300-600 lines, took about half a semester

Fall 2015

# Example projects with consensus in DistAlgo

| Algorithm | Description |
|---|---|
| Avalanche | Avalanche consensus protocol |
| BA* | Algorand Byzantine aggrement |
| SCP | Stellar consensus protocol |
| FaB | fixing Fast Byzantine consensus |
| VR | Viewstamped Replication |
| EVS | Extended Virtual Synchrony, and replication |
| Paxos-SB | Paxos for System Builders |
| Paxos-SB Perform | Paxos-SB performance eval, & cmp with C |
| Paxos-SB Flow | flow control and reconciliation in Paxos-SB |
| Derecho | fast SMR for cloud services |
| EPaxos | Egalitarian Paxos, & comparison with Go |
| Improving Raft | advanced Raft features and fixes |
| Leader Dynamic | leader election for dynamic netowrks |
| Epi Gossip | Epistemic gossip protocols |
| CRDT | conflict-free replicated data types |
| CRDT Edit | CRDT for collaborative editing |
| Improving Churn | garbage collection for churn algorithm |

Fall 2018, Spring 2019

and dozens of security protocols

# See DistAlgo run, and run it yourself!

get and run DistAlgo:

extremely easy, thanks to fabulous Bo Lin, and Python!

demo:

Basic Paxos, vRA Multi-Paxos, Bitcoin Backbone, ...
anything exciting you have if there is time or afterward
much more unofficial by students: failure injection, backup-restore, testing, debugging, visualization, ...

experimented with many well-known algorithms:

about all that we and hundreds of students found interesting

were helped significantly in teaching and research:

100+ different course projects at Stony Brook (incl. undergrads; and high school students who had done little or no programming).
external too: Jennifer Welch, Stephan Merz courses Spring 2019 so few bug found (none by external Spring 2019 users)

# Ongoing and future work

consensus algorithms

- better understanding and specification of more algorithms
- exact liveness properties and complexity guarantees

languages and compilers

- high-level features for simulation, testing, visualization,...
- many additional, improved analyses and optimizations

longer term

systematic design and optimization of distributed algorithms

Paxos, mutex, logical clock [PPDP19 to appear, APPLIED18, SSS12]
DistAlgo, compiler, optimization [TOPLAS17, PPDP16, OOPSLA12, ...]
formal verification of Paxos [FM 2016, NFM 2018, ...]

Thanks !